Load Test Report

Talo by Hevo · campus-hiring assessment

Platform: Cloudflare Worker + D1 + R2 · **Date:** 6 October 2026 · **Tool:** k6 v0.54 · **Target to prove:** ~100 students taking the assessment simultaneously

✓ Yes — we are good for 100 concurrent students. Verified on the live production system, with zero failures.

The full candidate journey (login → Part 1 with answer auto-saves + heartbeats → submit → Part 2 → submit) was tested end to end, two ways: on the real production deployment with 100 students at once, and locally, pushed far past the target to find the breaking point.

Test	Where	Students	Result	Failed reqs	p95 response
Production (real)	**talo.hevo-talo.workers.dev**	**100**	**✓ all 100 completed**	**0.00%**	**1.09 s**
Local — target	Local worst-case rig	100	✓ all completed	0.00%	28 ms
Local — 3×	Local worst-case rig	300	✓ all completed	0.00%	69 ms
Local — 5×	Local worst-case rig	500	✓ all completed	0.00%	541 ms
Local — stress	Local worst-case rig	~700–1,100	⚠ saturation point	14%	15.5 s

p95 = 95% of all responses were faster than this. On production, all 100 students completed with zero failed requests out of 1,800 calls, and 95% of responses returned within 1.1 seconds end-to-end. Locally — where there is no network delay, isolating pure compute + database — the system stays clean to 500 students (5× target) and only strains at 7–10×.

Production results — verified at 100 students

Run against the live deployment **talo.hevo-talo.workers.dev** from a laptop, using 100 throwaway test accounts that were seeded, run through the real flow, and deleted afterwards — no real candidate accounts touched, nothing left behind. This is the actual production stack: Cloudflare's edge, the real D1 database, and the live question bank.

Metric	Production · 100 students
Students completed	**100 / 100**
Total requests	1,800
Failed requests	**0.00% (0 of 1,800)**
Checks passed	**100% (600 of 600)**
Median response	613 ms
p95 response	**1.09 s**
p99 response	1.14 s
Slowest response	1.33 s

Every login, question load, answer auto-save, heartbeat and submission across all 100 students succeeded. Response times here include the full internet round-trip from the test laptop to Cloudflare, so they represent the **real end-to-end experience** a student has — not a lab figure. Zero requests failed.

What was actually tested

This is not a static-page ping test. Each simulated student ran the **complete, real flow**, which is also where the database does its heaviest work: log in, start Part 1 (which assigns their randomized question set), answer each question with **auto-save on every question** and periodic **heartbeats** (the timer sync), submit Part 1, then start, answer and submit Part 2.

That is ~18 server calls per student, and it includes **every database write** the live system makes during an exam: logins, answer auto-saves, and heartbeats. Database write contention is the one thing that genuinely gets harder as more students join at once — so that is exactly what we stressed. To stay conservative, all students in each run **arrived within a ~60-second window**, a denser burst than a real drive where students trickle in over several minutes.

Detailed local results (stress & headroom)

Metric	100 (target)	300	500	Stress peak
Students completed	100 / 100	300 / 300	500 / 500	501 ok, 827 failed
Total requests	1,800	5,400	9,000	18,148
Failed requests	**0.00%**	**0.00%**	**0.00%**	**14.0%**
Checks passed	100%	100%	100%	85%
p95 latency	28 ms	69 ms	541 ms	15.5 s
p99 latency	39 ms	110 ms	779 ms	21.3 s
Slowest response	65 ms	264 ms	1.56 s	25.4 s

Reading the stress column: we deliberately kept raising the arrival rate until the system broke, to find the ceiling. On the local test machine that happened around 700–1,100 students hammering simultaneously. Even

at that extreme the server **degraded gracefully** — requests queued and slowed, ~14% timed out, but the process did not crash and recovered as load dropped. That is the behaviour you want at the edge of capacity, and it is far beyond any realistic campus turnout.

Why this is conservative (production is stronger, not weaker)

The local numbers come from a **single process on one CPU core** with the database as a **single local SQLite file** — deliberately the worst case for write contention. Production is more capable in two ways that matter:

- **Cloudflare Workers auto-scale across isolates.** Each request runs in a lightweight isolate that Cloudflare distributes across its global fleet — there is no single-box concurrency cap to hit at 100 users.
- **Production D1 is a managed database service**, not a local SQLite file on one core. The real shared bottleneck in any such app is the database, and we stressed the harshest version of it locally — and it still cleared 500 students with zero errors.

So the local result is a **floor**, not a ceiling — and the production run confirmed the real edge handles the target cleanly. For reference, Cloudflare's documented Workers Paid / D1 limits (CPU 30 s–5 min, D1 up to 10 GB, 1,000 queries per request) sit far above what 100 students doing a few small writes each will ever touch.

How the production test was kept safe

The production run used **100 synthetic throwaway accounts** (`load-####`, deleted afterwards), not real candidates — so no real student's account or one-test-per-email lock was affected, and the database was returned to its exact prior state. It was run outside any live assessment window. The candidate flow never calls the AI grader or resume storage, so the test added no AI/storage cost — only the genuine database write load, which is exactly what we wanted to measure.

AI (OpenAI) usage & cost per student

Talo uses OpenAI for three one-time, admin-side jobs only — reading the resume, writing each student's personalized Part 2 questions, and grading. **The exam itself makes zero AI calls**, so how many students take the test at once has no effect on AI cost. Model in use: **gpt-4o-mini**.

Stage	When it runs	OpenAI calls	What the call does
Resume parse	On resume upload	**1**	Pulls out name, email, skills, projects — *all in this one call* (name + email are not separate)
Part 2 question generation	On resume upload	**1**	Writes both personalized scenario questions in a single call
Part 1 question assignment	When student starts	0	Chosen from the existing question bank — no AI
Taking the exam	During the test	**0**	Login, answers, auto-saves, heartbeats, submit — none call AI
Part 1 grading	When admin grades	**3**	One call per Part 1 question (2 SQL + 1 Python)
Part 2 grading	When admin grades	**2**	One call per Part 2 question
Total per student	**intake → graded**	**7 calls**	
Interview questions (optional)	Only if shortlisted	+1	Drafts interview questions from the resume + answers

The PDF-to-text step runs locally (no AI, no cost). Uploading the resume itself is plain file storage, not an AI call. Part 3 is retired, so it adds nothing. Grading is the heaviest part — 5 of the 7 calls.

Cost. At gpt-4o-mini pricing (**$0.15 per 1M input tokens, $0.60 per 1M output tokens**) and typical resume/answer sizes, each student costs roughly **$0.003–0.005 — under half a cent — for the entire pipeline** (resume → questions → grading).

Students	Approx. total OpenAI cost
100	~$0.30 – $0.50
500	~$1.50 – $2.50
1,000	~$3 – $5

For planning: (1) because the live exam uses no AI, a 100-student simultaneous drive costs the same in API as 100 students spread over a week — load does not drive AI spend; and (2) grading is the main cost and only runs when you choose to grade, so you control when that spend happens. Token sizes are estimates; actual spend is always visible on your OpenAI usage dashboard.

How to reproduce

Everything lives in the repo under `loadtest/`. Production 100-student run (seeds throwaway accounts, runs the full flow, cleans up):

```
# 1) seed 120 throwaway candidates on production
npx wrangler d1 execute campus-hiring-assessment-db --remote --file loadtest/prod-seed-100.sql
# 2) run the real 100-student spike against production
k6 run -e BASE_URL=https://talo.hevo-talo.workers.dev -e MODE=flow -e SCEN=spike -e POOL=120 \
  --summary-export loadtest/results/prod-spike-summary.json loadtest/candidate-flow.js
```

```
# 3) delete all throwaway data from production
npx wrangler d1 execute campus-hiring-assessment-db --remote --file loadtest/prod-cleanup.sql
```

Local runs (stress + headroom), no production impact:

```
bash loadtest/run-local.sh spike      # 100 students
bash loadtest/run-local.sh headroom   # 300 students
bash loadtest/run-local.sh mid500     # 500 students
bash loadtest/run-local.sh stress     # ramp until it breaks
```

In one line, for the manager

We load-tested the real production system with 100 students taking the full assessment at once: **all 100 finished, zero failures, responses under ~1.1 seconds**. We also pushed a local copy to 5× that load with no errors, and it only strained at 7–10×. **Talo is ready for the campus drive.**

Cloudflare platform notes: Workers & D1 limits (fabricate.build), Cloudflare Workers limits 2026 (markaicode.com). Confirm exact plan limits at developers.cloudflare.com. · Prepared with k6 load testing · Talo by Hevo.